



Faculty of Engineering and Technology

Computer Science Department

Software Engineering

COMP433

Software Requirement Specifications and Design-SRSD

Business Title: Cake Shop



Instructor: Prof. Adel Taweel

Sec:3

Date:25.6.2026

Chapter 1: Project Planning and Management

1.1–1.5. Team Overview and Business Title

1.6. Project management strategy

1.7. Team Pictures

1.8. Group members report

Chapter 2: Requirement Elicitation, Analysis and Modelling

2.1. Business description (Phase 1)

1. Overview

2. Key Features and Services

3. Business Capacities

4. Business Operations

5. Stakeholders

6. Environment and Challenges

7. Conclusion

2.2. User and System requirements

User Requirement

System Requirements

2.3. Effort/Time estimation calculation

2.4. Actors analysis and description

2.5. Overall use-case Diagram

2.6. Key Use-Cases Description

2.7. Main Activity Diagram

Chapter 3: System Analysis and Modelling

3.1. System CLASS Diagrams

Analysis Class Model

Detailed Class Model

3.2. Sequence Diagrams

Chapter 4: System Design and Modelling

4.1. Description of chosen Design Goals

General Goals

Specific Goals

4.2. Component Diagram

4.3. Overall architecture diagram

4.4. Deployment diagram

Appendix-I: Minutes of Meetings

Chapter 1: Project Planning and Management

1.1-1.5. Team Overview and Business Title

Report and Phases Writer: Sama Baraawi

Business Title: Cake Shop

Group Number: G2

Members:

Sama Baraawi	1231022	QA & Software Tester
Nawras Yacoob	1221904	Requirement Engineer
Majd Nazzal	1230071	Project Manager
Jwan Rajha	1230662	Secretary
Mohammad Abutteen	1221573	Technical Architect

1.6. Project management strategy

The project was managed through a structured and collaborative approach that ensured effective coordination among team members throughout the development lifecycle. Regular team meetings were conducted on a weekly basis, with additional meetings arranged when necessary to discuss project progress, resolve issues, and coordinate upcoming tasks. Communication was maintained continuously through online channels to facilitate timely collaboration and information sharing.

Project responsibilities were distributed among team members according to the requirements of each project phase. Progress was monitored during team meetings, where completed work was reviewed, pending tasks were identified, and action plans were established for subsequent activities.

Decision-making was primarily based on group discussion and consensus. Major project decisions, including design choices and system modeling approaches, were collectively evaluated by team members to ensure that different perspectives were considered before reaching a final agreement. This collaborative approach promoted active participation and accountability within the team.

The team adopted a hybrid software development strategy that combined the strengths of both the **Waterfall** and **Agile** process models. The overall project followed the sequential phases of the Waterfall model, progressing from business description and requirements analysis to system analysis, system design, and final documentation. At the same time, Agile principles were incorporated by performing continuous reviews, iterative refinements, and frequent updates based on instructor feedback and internal evaluations. This hybrid approach enabled the team to maintain a well-organized development process while remaining flexible enough to accommodate improvements, ensuring that the final system design achieved a high level of quality, consistency, and completeness.

1.8. Group members report

Sama Baraawi - QA & Software Tester

From my perspective, I consider our project to be highly successful because throughout the semester we continuously improved our work by incorporating the instructor's feedback after every phase, which significantly enhanced the quality, consistency, and completeness of the final deliverables. I was highly committed to the project and invested considerable time and effort to ensure that every deliverable met the required quality standards. In Phase 1, I wrote the initial Business Description, presented it to the team, discussed it with them, and finalized it after incorporating their suggestions. In Phase 2, I took the lead on all assigned tasks and completed the required deliverables while collaborating with the team. In Phase 3, I led the Actor Analysis task and actively contributed to the Use-Case Modelling and Activity Modelling tasks through reviewing, discussing, validating, and finalizing the diagrams. In Phase 4, I led the System Design Goals task, participated in redesigning and improving the Deployment Diagram, and also reviewed, discussed, and verified both the Analysis Class Diagram and the Detailed Class Diagram, ensuring the correctness of the classes, relationships, multiplicities, and overall consistency. I also reviewed and discussed the Architecture Diagram and the Component Diagram to ensure they complied with UML principles and the course guidelines. Throughout the project, I carefully followed all of the instructor's comments after each submission and was responsible for applying the required modifications before the next phase was submitted. In addition, I collected all tasks together, organized them into the required report structure, prepared and submitted the PDF for every project phase, and prepared the project presentation. Overall, I believe I carried a significant share of the project's technical review, documentation, quality assurance, and coordination activities, and this experience greatly strengthened my software engineering knowledge, UML modelling skills, teamwork, attention to detail, and ability to produce a professional and well-structured software design project.

Nawras Yacoob -Requirement Engineer

In my opinion, our project turned out to be successful as we consistently worked on improving it throughout the semester by applying the instructor's feedback after each phase, which led us to achieve a clear and well-organized final system design. I actively participated in the entire project by reviewing deliverables, discussing ideas with my teammates, and providing feedback during all project phases. I was responsible for designing the **Use Case Diagram** and the **Component Diagram**, ensuring that they accurately reflected the system requirements and UML principles. In addition, I played an important role in maintaining effective communication within the team by coordinating discussions, encouraging collaboration, and supporting my teammates whenever needed. I always made every effort to contribute to all tasks and remain actively involved throughout the project, which helped strengthen our teamwork and the overall quality of the final deliverables.

Mohammad Abutteen – Technical Architect

I believe our project achieved its objectives successfully because our team worked collaboratively throughout the semester and continuously refined the deliverables based on the instructor's feedback. This iterative process helped us improve both the quality and consistency of the final system design. My main contributions were during Phases 3 and 4, where I took the lead in designing the **Activity Diagram**. I revised and refined it several times to ensure it fully satisfied the instructor's requirements and accurately represented the system workflow. I also designed the **Architecture Diagram**, making sure it reflected the system structure and supported the selected design goals. In addition, I actively participated in reviewing deliverables, discussing design decisions with the team, and contributing to the overall development of the project, helping us produce high-quality final deliverables.

Jwan Rajha – Secretary

Our **Cake Shop System** Software Engineering project was a valuable learning experience, and I believe it was successful because our team worked together effectively and continuously improved the project based on the instructor's feedback. As the **Secretary**, I contributed throughout all project phases by organizing the team's work, participating in discussions, reviewing project deliverables, and helping ensure that the documentation was complete and consistent. My main contribution was leading **Task 4.6 (Deployment Diagram)** during the final phase, where I designed and refined the deployment model. I also contributed to reviewing diagrams and coordinating the final report. I am proud of the teamwork, commitment, and progress we achieved throughout the **Cake Shop System** project.

Majd Nazzal – Project Manger

I believe our **Cake Shop System** project was successful because our team worked collaboratively throughout all project phases and continuously improved the system based on the instructor's feedback. This project helped me strengthen my understanding of software engineering concepts, UML modeling, and system design. I contributed to all phases of the project by participating in discussions, reviewing requirements, and refining different project deliverables. My main contributions were in Task 4.1 (Detailed Class Diagram) and Task 4.2 (Sequence Diagram – Assign Order to Staff), where I led the design, development, and revision of these diagrams. I also participated in reviewing other diagrams and ensuring consistency across the final project report.

Chapter 2: Requirement Elicitation, Analysis and Modelling

2.1. Business description (Phase 1)

MODIFIED By Sama Baraawi

Business Description Outline – Dentist Clinic

A dental clinic is a healthcare business that provides oral healthcare services to patients of all ages. Its primary function is to diagnose, treat, and prevent oral and dental diseases while maintaining patients' overall oral health. The clinic offers a wide range of services, including routine dental examinations, professional teeth cleaning, tooth fillings, tooth extractions, root canal treatment, gum disease treatment, cosmetic dental procedures, emergency dental care, and follow-up visits. In addition to providing dental treatment, the clinic manages patient registration, appointment scheduling, patient records, and payment collection as part of its daily business operations. The clinic is staffed by dentists, receptionists, administrative personnel, and dental assistants who work together to provide continuous and efficient patient care. Currently, all patient records, appointments, and administrative activities are managed manually using paper-based files and registers.

The clinic provides several business services that support its daily operations and patient care:

1. Patient Registration Service

This service is provided to new patients visiting the clinic for the first time. The receptionist collects the patient's personal information, contact details, medical history, allergies, and other relevant information. A paper-based patient file is then created to document the patient's information, while returning patients have their existing paper file retrieved and updated with any changes before receiving treatment.

2. Appointment Booking Service

This service allows patients to schedule appointments for dental examinations or treatments. The patient contacts the clinic or visits the reception desk to request an appointment and specifies the required dental service. The receptionist checks the dentists' schedules, identifies an available time slot with the appropriate dentist based on the required treatment, records the appointment in the appointment register, and confirms the appointment with the patient.

3. Appointment Rescheduling Service

This service is provided when a patient requests to change an existing appointment. The receptionist checks the dentists' schedules for an alternative available time, updates the appointment record with the new date and time, and confirms the rescheduled appointment with the patient.

4. Appointment Cancellation Service

This service is provided when a patient decides not to attend a scheduled appointment. The patient informs the clinic before the appointment whenever possible. The receptionist cancels the appointment in the appointment register and makes the time slot available for other patients.

5. Consultation and Examination Service

This service is provided to evaluate the patient's oral health and identify dental problems before any treatment is performed. During the consultation, the dentist reviews the patient's medical and dental history, discusses the patient's symptoms or concerns, performs a comprehensive oral examination, and may request diagnostic procedures such as dental X-rays when necessary. Based on the

examination results, the dentist identifies the dental condition, explains the diagnosis to the patient, recommends the appropriate treatment options, and records the examination findings in the patient's paper-based record.

6. Dental Treatment Service

This service provides the clinical procedures required to treat diagnosed dental conditions and restore the patient's oral health. Depending on the patient's diagnosis, the clinic offers professional teeth cleaning, tooth fillings for dental cavities, tooth extractions, root canal treatment, gum disease treatment, minor restorative procedures, and preventive dental care. Before starting treatment, the dentist explains the recommended procedure and obtains the patient's consent. After completing the treatment, the dentist records all treatment details, materials used, and any follow-up instructions in the patient's paper-based record.

7. Emergency Dental Care Service

This service is provided to patients who require immediate dental care due to conditions that cannot wait for a scheduled appointment, such as severe tooth pain, uncontrolled bleeding after dental trauma, facial swelling caused by infection, dental abscesses, broken or knocked-out teeth, or other acute dental emergencies. Upon arrival, the dentist immediately examines the patient to determine the severity of the condition. If emergency treatment is required, the patient is given priority over routine appointments, and non-urgent appointments may be delayed when necessary. The dentist then provides immediate treatment to relieve pain, control the emergency condition, or stabilize the patient before arranging any additional treatment or follow-up visit if required.

8. Payment Service

This service is responsible for collecting payments after dental treatment is completed. The total treatment cost is determined based on the type of dental procedure performed, the number of procedures provided during the visit, and any additional materials or services used during treatment. The receptionist informs the patient of the total amount due, accepts payment by cash or card, issues a payment receipt, and records the transaction for financial documentation.

9. Patient Record Management Service

This service is responsible for maintaining complete paper-based patient records throughout the patient's treatment history. Each patient file contains personal information, medical history, dental history, diagnoses, treatment plans, procedures performed, prescribed medications when applicable, radiographic findings, and follow-up recommendations. After every patient visit, the dentist updates the patient's paper record with the latest examination findings and treatment details to ensure continuity of care during future visits.

Business Capacity

The clinic consists of **5 dental treatment rooms**, each equipped with one dental chair and the necessary clinical equipment to perform different dental procedures. It employs **5 dentists, 2 receptionists, 1 administrative staff member, and 3 dental assistants**. The receptionists are responsible for patient registration, appointment scheduling, payment collection, and maintaining paper-based patient records. The administrative staff member oversees the clinic's daily

administrative and operational activities, while dental assistants prepare treatment rooms, sterilize instruments, assist dentists during procedures, and support patient care.

The clinic offers approximately **15 different dental services**, serves an average of **450 patients per month**, and handles around **600 appointments per month**, including routine visits, follow-up appointments, and emergency cases. On average, the clinic receives **20–25 patients per working day**, with the highest patient demand occurring during the afternoon and evening hours. These operational resources and service capacity enable the clinic to provide timely and continuous dental care while maintaining efficient daily operations.

How the Business Works

The clinic follows a structured workflow that begins when a new patient registers at the reception desk or a returning patient's paper-based record is retrieved. The receptionist schedules an appointment according to the availability and specialization of the appropriate dentist. During the visit, the dentist performs the consultation, establishes a diagnosis, and provides the required dental treatment. After treatment is completed, the receptionist collects the payment and schedules a follow-up appointment if required.

The clinic's primary source of revenue is the fees charged for dental services provided to patients. Treatment fees vary according to the type and complexity of the procedure, such as routine examinations, fillings, root canal treatment, cosmetic procedures, or emergency care. The clinic's revenue depends on the number of patients served and the range of dental services provided.

Stakeholders

The clinic's primary stakeholders include patients, dentists, receptionists, dental assistants, administrative staff, and clinic management. Patients seek safe and effective dental care, dentists are responsible for diagnosing and treating oral health conditions, receptionists manage patient registration, appointment scheduling, and payment collection, while dental assistants support dentists during treatment and prepare clinical equipment. Administrative staff oversee daily administrative operations, and clinic management is responsible for supervising the clinic, allocating resources, maintaining service quality, and ensuring efficient business performance. Together, all stakeholders collaborate to ensure efficient clinic operations and high-quality patient care.

Environment and Challenges

The clinic operates in a healthcare environment where patient safety, treatment quality, appointment availability, and efficient use of clinical resources are critical to daily operations. The clinic experiences fluctuations in patient demand throughout the week, with higher workloads during peak periods. Limited treatment rooms, unexpected emergency cases, patient cancellations or missed appointments, maintaining accurate paper-based records, and coordinating dentists' schedules all create operational challenges that directly affect productivity, patient waiting times, and service quality. These challenges require effective resource allocation, staff coordination, and appointment management to maintain high-quality patient care.

Conclusion

The dental clinic is a healthcare business that provides a wide range of oral healthcare services through coordinated teamwork and structured business processes. Its daily operations depend on effective appointment management, accurate patient records, qualified healthcare professionals, and efficient coordination to deliver safe and high-quality dental care.

2.2. User and System requirements

User Requirement:

Lead: Sama Baraawi

Contributors: [Sama Baraawi: discussion, documentation], [Nawras Yacoob: discussion], [Jwan Rajha: documentation], [Majd Nazzal: reviewing]

MODIFIED By Sama Baraawi

UR1.0: The customer shall be able to view available cake products.

UR2.0: The customer shall be able to place a standard cake order.

UR3.0: The customer shall be able to request a custom cake order.

UR4.0: The customer shall be able to choose between home delivery and in-store pickup for an order.

UR5.0: The customer shall be able to select a payment method and complete payment for an order.

UR6.0: The customer shall be able to track the status of a placed order.

UR7.0: The customer should be able to submit a review for a completed order.

UR8.0: Authorised staff shall be able to accept customer orders.

UR9.0: Authorised staff shall be able to update the status of customer orders.

UR10.0: Authorised staff shall be able to add, update, and remove cake products.

UR11.0: Authorised staff shall be able to add, update, and remove promotional offers.

UR12.0: Branch managers shall be able to generate sales and order reports.

System Requirements:

Lead: Sama Baraawi

Contributors: [Sama Baraawi: discussion, documentation], [Nawras Yacoob: discussion], [Jwan Rajha: discussion], [Majd Nazzal: reviewing]

MODIFIED By Sama Baraawi

UR1.0: The customer shall be able to view available cake products.

SR1.1: The system shall provide information about available cake products, including flavour, size, price, available customisation options, and availability status.

SR1.2: The system shall allow customers to filter available cake products by category, flavour, and price range.

SR1.3: The system shall return filtering results based on the selected criteria.

UR2.0: The customer shall be able to place a standard cake order.

SR2.1: The system shall allow the customer to add an available cake product to an order.

SR2.2: The system shall allow the customer to enter a name and phone number before confirming the order.

SR2.3: The system shall allow a delivery address when home delivery is selected.

SR2.4: The system shall allow the customer to select a pickup branch and pickup time when in-store pickup is selected.

SR2.5: The system shall allow the customer to review the order details before confirming the order.

SR2.6: The system shall generate a unique order identifier for each confirmed order.

SR2.7: The system shall make the details of confirmed orders available for order tracking.

SR2.8: The system shall notify the customer when mandatory order information is incomplete before the order is confirmed.

UR3.0: The customer shall be able to request a custom cake order.

SR3.1: The system shall allow the customer to specify the flavour, size, design description, cake message.

SR3.2: The system shall allow the customer to upload a reference image for a custom cake design.

SR3.3: The system shall allow the customer to select a delivery or pickup date that is at least 48 hours after the order request.

SR3.4: The system shall notify authorised staff when a new custom cake order is submitted.

UR4.0: The customer shall be able to choose between home delivery and in-store pickup for an order.

SR4.1: The system shall allow the customer to choose either home delivery or in-store pickup for an order.

SR4.2: The system shall allow the customer to specify the delivery address when home delivery is selected.

SR4.3: The system shall allow the customer to select a pickup branch, pickup date, and pickup time when in-store pickup is selected.

SR4.4: The system shall calculate the delivery fee according to the selected delivery location and the predefined delivery charges for that location.

UR5.0: The customer shall be able to select a payment method and complete payment for an order.

SR5.1: The system shall provide the payment methods available for the selected order type and fulfilment option.

SR5.2: The system shall allow the customer to confirm the selected payment method before completing the order.

SR5.3: The system shall record the payment status of each order as pending, partially paid, or paid.

SR5.4: The system shall support full payment for standard cake orders and a 50% deposit for custom cake orders before order confirmation.

SR5.5: The system shall notify the customer when the required payment amount has not been completed before order confirmation.

SR5.6: The system shall allow payment only for active orders.

UR6.0: The customer shall be able to track the status of a placed order.

SR6.1: The system shall allow the customer to view the current status of a placed order.

SR6.2: The system shall identify each order as pending, confirmed, preparing, ready for pickup, out for delivery, delivered, cancelled, or rejected.

SR6.3: The system shall make the updated order status available to the customer after it is changed by authorised staff.

UR7.0: The customer should be able to submit a review for a completed order.

SR7.1: The system should allow the customer to submit a rating and written feedback for a completed order.

SR7.2: The system shall associate each review with the corresponding completed order.

SR7.3: The system shall allow authorised staff to view customer reviews.

UR8.0: Authorised staff shall be able to accept customer orders.

SR8.1: The system shall allow authorised staff to view pending customer orders.

SR8.2: The system shall allow authorised staff to accept customer orders.

SR8.3: The system shall change the order status from **Pending** to **Confirmed** after the order is accepted.

SR8.4: The system shall notify the customer that the order has been confirmed.

UR9.0: Authorised staff shall be able to update the status of customer orders.

SR9.1: The system shall allow authorised staff to view confirmed customer orders.

SR9.2: The system shall allow authorised staff to assign confirmed orders to preparation or delivery staff.

SR9.3: The system shall allow authorised staff to update the order status throughout the fulfilment process.

SR9.4: The system shall notify the customer whenever the order status is updated.

UR10.0: Authorised staff shall be able to add, update, and remove cake products.

SR10.1: The system shall allow authorised staff to add new cake products.

SR10.2: The system shall allow authorised staff to update the details of existing cake products.

SR10.3: The system shall allow authorised staff to remove cake products that are no longer available.

SR10.4: The system shall allow authorised staff to enter the flavour, size, price, category, and availability status of each cake product.

UR11.0: Authorised staff shall be able to add, update, and remove promotional offers.

SR11.1: The system shall allow authorised staff to add new promotional offers.

SR11.2: The system shall allow authorised staff to update promotional offers.

SR11.3: The system shall allow authorised staff to remove promotional offers.

SR11.4: The system shall apply active promotional offers to eligible cake products or customer orders.

UR12.0: Branch managers shall be able to generate sales and order reports.

SR12.1: The system shall allow branch managers to select either a sales report or a customer order report.

SR12.2: The system shall allow branch managers to specify the reporting period.

SR12.3: The system shall generate the selected report.

SR12.4: The generated report shall display the relevant information, including total sales, completed orders, cancelled orders, and pending orders, where applicable.

2.3. Effort/Time estimation calculation

Lead: Sama Baraawi

Contributors: [Sama Baraawi: discussion, documentation], [Nawras Yacoob: discussion], [Jwan Rajha: reviewing], [Majd Nazzal: reviewing]

MODIFIED By Sama Baraawi

Effort Estimation Table:

UR	Estimated No. of Developers can work concurrently on UR	Estimated Effort (average/developers)	Total Effort (for One Developer)
UR1	2	2 pw	$2 * 2 = 4$ pw
UR2	2	4 pw	$4 * 2 = 8$ pw
UR3	3	4 pw	$4 * 3 = 12$ pw
UR4	2	2 pw	$2 * 2 = 4$ pw
UR5	2	3 pw	$3 * 2 = 6$ pw
UR6	1	2 pw	$2 * 1 = 2$ pw
UR7	1	2 pw	$2 * 1 = 2$ pw
UR8	2	2 pw	$2 * 2 = 4$ pw
UR9	2	4 pw	$4 * 2 = 8$ pw
UR10	2	4 pw	$4 * 2 = 8$ pw
UR11	2	3 pw	$3 * 2 = 6$ pw
UR12	1	3 pw	$3 * 1 = 3$ pw
Total effort / avg	$(2+2+3+2+2+1+1+2+2+2+2+1) / 12 = 1.83$ developers on average needed	35 pw	67 pw

Schedule Time Calculation (For one Developer)

Schedule time (minimum time to complete) = Total estimated effort with 30% buffer = 35×1.30
= 45.5 weeks

Schedule time (maximum time to complete) = Total effort for one developer x 30% buffer = 67×1.30 = 87.1 weeks

The development team consists of five developers working in parallel.

Minimum schedule time = Total estimated effort $\times$ 30% buffer $\div$ Number of developers
= $35 \times 1.30 \div 5$ = 9.1 weeks

Maximum schedule time = Total effort $\times$ 30% buffer $\div$ Number of developers
= $67 \times 1.30 \div 5$ = 17.42 weeks

Cost estimation:

Average salary/week = \$250.

Base cost = 250×67 = \$16,750.

Minimum cost with 10% profit margin = $16,750 \times 1.10 = \$18,425$.

Maximum cost with 30% profit margin = $16,750 \times 1.30 = \$21,775$.

Final Estimation Summary:

Item	Value
Minimum schedule time	9.1 weeks
Maximum schedule time	17.42 weeks
Base cost	\$16,750
Minimum cost	\$18,425
Maximum cost	\$21,775

Agreed Cost and Time with the Customer

During the initial meeting held with the customer group in class, both parties agreed on a project cost of **\$10,000** and a development schedule of **20 weeks**. However, after the requirements were reviewed and refined, several changes and additional functionalities were introduced, which increased the estimated development effort and overall project cost.

As a result, a second meeting was held with the customer group to discuss the impact of these changes. Following the discussion, both parties agreed to revise the project agreement. The final agreed cost was **\$20,000**, and the agreed project duration was extended to **22 weeks** to accommodate the additional requirements and the increased development effort.

The estimated project schedule indicates that the development work can be completed within **20 weeks (approximately 5 months)**. This duration is considered suitable for completing the project within the academic semester while allowing sufficient time for implementation, testing, revisions, and final delivery. Throughout the project, all **five developers** in the development team were actively utilized by assigning different user requirements and development tasks in parallel. This parallel development approach optimized the development process, reduced the overall project schedule, and ensured that the workload was distributed efficiently among all team members.

2.4. Actors analysis and description

Lead: Sama Baraawi

Contributors: [Sama Baraawi: discussion, documentation], [Nawras Yacoob: reviewing], [Jwan Rajha: reviewing]

MODIFIED By Sama Baraawi

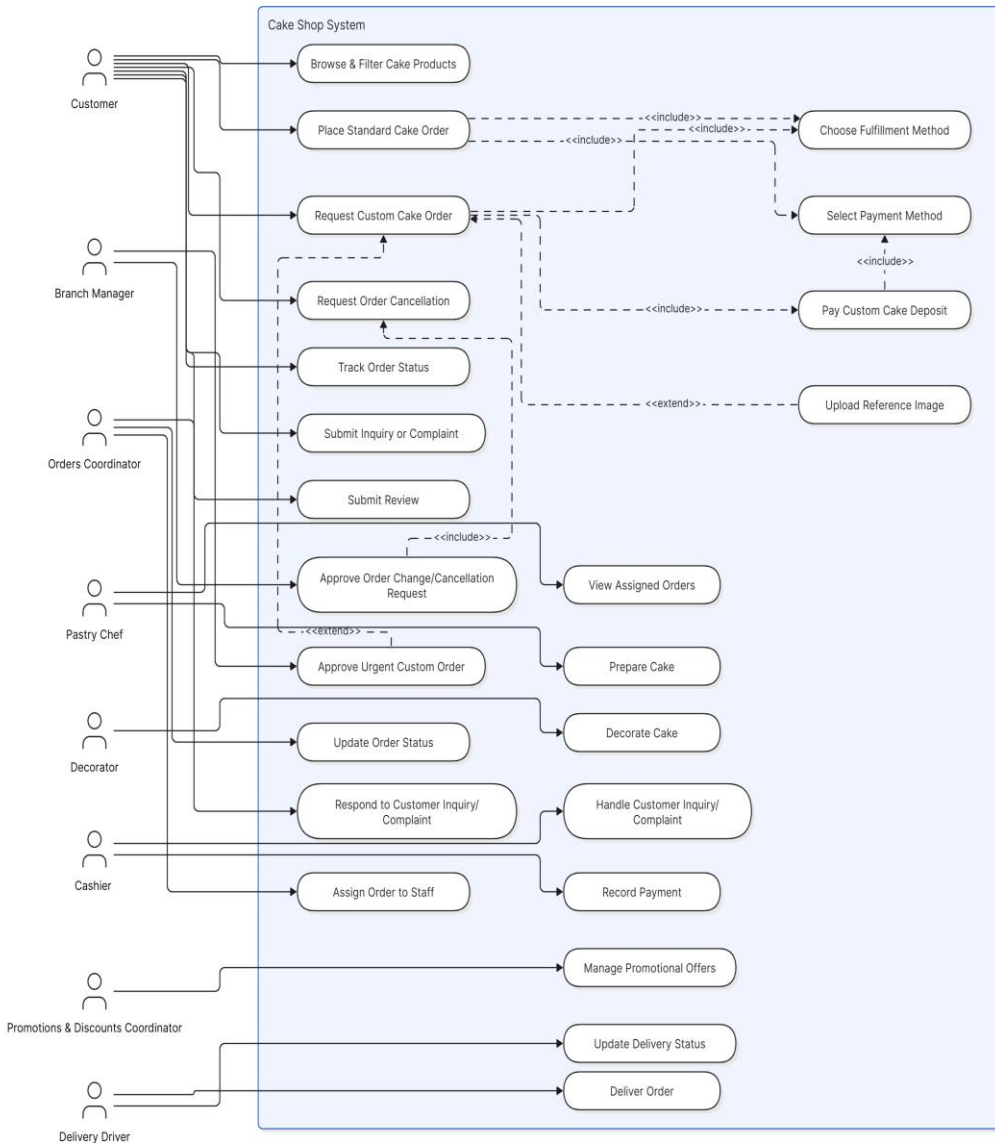
Actor	Semantics/Description
Customer	This actor represents someone who uses the online cake shop system to browse available cake products, place standard or custom cake orders, choose delivery or pickup options, select payment methods, track order status, communicate with the shop regarding order-related issues, and submit inquiries, complaints, or reviews after order completion.
Orders Coordinator	This actor represents a member of the cake shop staff who uses the system to view incoming customer orders, assign orders to staff members, update order statuses, and respond to customer inquiries and complaints according to assigned business responsibilities.
Branch Manager	This actor represents a member of the cake shop staff who uses the system to review order processing activities, approve urgent custom cake orders, approve order modification or cancellation requests, add, update, and remove cake products, generate sales and order reports, and supervise branch operations.
Pastry Chef	This actor represents a member of the cake shop staff who uses the system to view assigned cake orders, access custom cake requirements, and update the preparation status of assigned orders.
Decorator	This actor represents a cake shop staff member responsible for decorating custom cakes. The Decorator uses the system to access assigned custom cake orders, review the customer's decoration requirements, complete the cake decoration, and update the order status upon completion.
Cashier	This actor represents a member of the cake shop staff who uses the system to record customer payments, update payment status, respond to customer inquiries and complaints, and provide order-related assistance.
Delivery Driver	This actor represents a member of the cake shop staff who uses the system to view assigned delivery orders, update delivery status, and confirm completed deliveries.
Promotions & Discounts Coordinator	This actor represents a member of the cake shop staff who uses the system to add, update, remove, and apply promotional offers and discounts.

2.5. Overall use-case Diagram

Lead: Nawras Yacoob

Contributors: [Nawras Yacoob: drawing], [Jwan Rajha: reviewing], [Sama Baraawi: reviewing, discussion, finalizing]

MODIFIED By: Sama Baraawi



Note: A shareable link for this diagram could not be provided because the diagram was created using **Visual Paradigm**, which requires a paid subscription to export or publish diagrams online. Therefore, a screenshot of the final diagram has been included instead

2.6. Key Use-Cases Description

Name: Sama Baraawi

MODIFIED

System Name: **Cake Shop System**

Use Case Title	Place Standard Cake Order
Description	A customer places an order for a standard cake available in the cake catalog. The customer selects the cake, chooses a fulfillment method, completes payment, and receives order confirmation.
Actors	Customer.
Data	Customer Information, Cake Information, Order Information, Payment Information.
Stimulus / Trigger	Customer initiates a standard cake order request.
Pre-conditions	1. The Cake Shop System is available and operational. 2. The cake catalog is available in the system.
Workflow OR Sequence / Flow of Events	Main Success Scenario 1. The customer selects a standard cake from the cake catalog. [E1: Cake Unavailable] 1.1 The selected cake is unavailable. 1.2 The system informs the customer that the cake cannot be ordered. 2. The system displays the selected cake details. 3. The customer chooses a fulfillment method. [A1: Delivery Order] 3.1 The customer selects delivery. 3.2 The customer enters the delivery address.

- 3.3 The system calculates the delivery fee.
- 3.4 The system displays the order total.

[A2: Pickup Order]

- 3.5 The customer selects pickup.
- 3.6 The customer selects a pickup branch.
- 3.7 The customer selects a pickup date.
- 3.8 The customer selects a pickup time.

- 4. The customer selects a payment method.
- 5. The customer submits the order.

[E2: Payment Failed]

- 5.1 The payment is rejected.
- 5.2 The system displays a payment failure message.

- 6. The system processes and approves the payment.
- 7. The system creates the order.
- 8. The system assigns a unique order ID.
- 9. The system confirms the order.
- 10. The customer receives the order confirmation.

Post-conditions /
Response

- 1. A new standard cake order is created and stored in the system.
- 2. The order status is set to Confirmed.
- 3. A unique order ID is assigned to the order.
- 4. The successful payment transaction is recorded in the system.

Comments

Payment approval is required before the order can be confirmed.

Name: Jwan Rajha

MODIFIED

System Name: Cake Shop System	
Use Case Title	Deliver Order
Description	This use case allows the Delivery Driver to deliver an assigned customer order. The system records delivery completion details, updates the delivery record, and changes the order status upon successful delivery.
Actors	Primary Actor: Delivery Driver
Data	· Order Information · Customer Information · Delivery Address · Delivery System · Delivery Completion Information
Stimulus / Trigger	The Delivery Driver selects an assigned delivery order and starts the delivery process.
Pre-conditions	• The Delivery Driver is logged into the system. • The selected order status is "Ready for Delivery". • The selected order has been assigned to the Delivery Driver. • Delivery information has been recorded for the selected order.
Workflow / Sequence of Events	1. The Delivery Driver opens the assigned deliveries list. 2. The system displays all assigned delivery orders. 3. The Delivery Driver selects a delivery order. 4. The system displays customer and delivery information. 5. The Delivery Driver starts the delivery process. 6. IF the customer receives the order successfully THEN 6.1 The Delivery Driver delivers the order. 6.2 The customer receives the order. 6.3 The Delivery Driver confirms delivery completion. 6.4 The system records delivery completion details. 6.5 The system changes the order status to "Completed".

	ELSE 6.6 The system records the failed delivery attempt. 6.7 The system displays available delivery handling options.
Alternative / Error Flows	A1. Customer Unavailable 1. The Delivery Driver arrives at the delivery location. 2. The customer is unavailable to receive the order. 3. The Delivery Driver records the failed delivery attempt. 4. The order remains in "Ready for Delivery" status. A2. Incorrect Delivery Address 1. The Delivery Driver determines that the delivery address is incorrect. 2. The Delivery Driver contacts the customer. 3. The customer provides the correct delivery address. 4. The delivery process continues. A3. Order Cancelled Before Delivery 1. The system detects that the order has been cancelled. 2. The delivery process is stopped. 3. The order status remains "Cancelled".
Post-conditions	Success • The selected order status is "Completed". • Delivery completion information is recorded in the system. • The delivery record has been updated. • The order is marked as delivered. Failure • The selected order remains in "Ready for Delivery" status. • The failed delivery attempt is recorded. • The order is awaiting a new delivery attempt.

Comments	Only authorised Delivery Drivers may access assigned delivery orders and confirm delivery completion. All delivery activities are recorded by the system for tracking and auditing purposes.
----------	--

MODIFIED

System Name: Cake Shop System	
Use Case Title	Manage Promotional Offers
Description	A Promotions & Discounts Coordinator may create, update, or deactivate promotional offers for cake products. The system validates offer details, saves the promotion, and automatically applies active offers to eligible customer orders during checkout.
Actors	Promotions & Discounts Coordinator (primary actor)
Data	Promotion name, discount type (percentage/fixed), discount value, eligible cake categories, specific cakes, start date, end date, active/inactive status
Stimulus / Trigger	The Promotions & Discounts Coordinator selects the “Manage Promotional Offers” option from the staff management menu.
Pre-conditions	• The cake shop system is operational and the Promotional Offers management module is accessible. • The Coordinator's account exists in the system with valid administrative privileges to manage offers. • The cake catalog (categories and individual cakes) is available in the system for offer association.
Workflow / Sequence of Events	1. The Coordinator navigates to the “Promotional Offers” management section. 2. The system displays the list of existing offers (active and inactive). 3. The Coordinator selects an action: Create, Update, or Deactivate. IF the Coordinator selects “Create” (Normal Path): 4. The Coordinator enters offer details: name, discount type, discount value, eligible cakes/categories, start date, and end date. 5. The system validates that all required fields are filled. 6. IF the discount value is positive AND the end date is after the start date AND there is no overlapping active offer for the same products: • 6.1 The system saves the new promotional offer with status “active”. • 6.2 The system displays a success confirmation message. IF validation fails → go to Alternative Flow C (Validation Errors). IF the Coordinator selects “Update” (Alternative Flow A): 7. The Coordinator selects an existing offer from the list. 8. The system displays the current offer details.

	9. The Coordinator edits the discount value, eligible products, or validity dates. 10. The system validates the updated details (same rules as step 6). IF valid → the system saves the changes and displays a success confirmation message. IF invalid → go to Alternative Flow C (Validation Errors). IF the Coordinator selects “Deactivate” (Alternative Flow B): 11. The Coordinator selects an active offer from the list and clicks “Deactivate”. 12. The system asks the Coordinator to confirm the deactivation. 13. IF confirmed: the system changes the offer status to “inactive” and stops applying it to new orders, then displays a success confirmation message. IF cancelled: the offer remains unchanged and the use case returns to step 2.
Alternative Flow C — Validation Errors (business rule violations)	C.1 Invalid dates: The end date is not after the start date. The system displays: “End date must be after start date.” The Coordinator must correct the dates and resubmit. C.2 Overlapping offer: The discount conflicts with an existing active offer on the same product(s). The system displays a warning. The Coordinator may adjust the dates, discount value, or eligible products and resubmit.
Post-conditions	• IF Create is successful: a new Promotion record exists in the system with status “active”, linked to its eligible cakes/categories and validity dates. • IF Update is successful: the existing Promotion record's attributes are updated and persisted with the new values. • IF Deactivate is successful: the Promotion record's status is set to “inactive”, and the system excludes it from future order calculations. • IF validation fails at any point: no changes are persisted, and the Promotion record (if existing) retains its previous state.
Comments	Only an authorised Promotions & Discounts Coordinator may manage promotional offers (SR9.1–SR9.3). Promotions can be percentage-based or fixed amount. The automatic application of active offers to eligible customer orders during checkout (SR9.4) is an internal system behaviour triggered at order time, not a direct interaction performed by an actor within this use case — it is therefore not modelled as a separate actor here. Technical/infrastructure failures (e.g., server or database errors) are handled by general system-level error handling and are out of scope for this business-level use case description.

Name: Nawras Yacoob

MODIFIED

System Name: Cake Shop System	
Use Case Title	Request Custom Cake Order
Description	A Customer may request a custom cake by specifying cake details (flavour, size, design description, cake message), uploading a reference image, selecting a delivery or pickup date/time, reviewing and approving the design, and paying a required 50% deposit before the order is confirmed.
Actors	Customer (primary actor), Branch Manager (secondary actor — involved only in the urgent-order approval path)
Data	Flavour, size, design description, cake message, reference image, requested date/time, urgency flag, deposit amount, payment status, order status
Stimulus / Trigger	The customer selects the “Request Custom Cake Order” option from the cake shop system’s main order menu.
Pre-conditions	• The cake shop system is operational and accessible. • The custom cake ordering option is enabled and available in the system. • The system is connected to a functioning payment gateway.
Workflow OR Sequence / Flow of Events	1. The customer selects the “Request Custom Cake Order” option. 2. The customer enters the cake flavour, size, design description, and cake message. 3. The customer uploads a reference image of the desired design. 4. The customer selects a preferred delivery or pickup date and time. 5. The system checks the selected date/time against availability and the 48-hour minimum rule. 6. IF the date is valid (≥ 48 hours ahead) and the slot is available (Normal Path): - o 6.1 The system displays a preview of the custom request for customer review. - o 6.2 The customer reviews the design. - o 6.3 IF approved → proceed to step 6.4. IF not satisfied → go to Alternative Flow A (Design Modification). - o 6.4 The system calculates the 50% deposit based on cake size, flavour, and design complexity. - o 6.5 The customer selects a payment method and submits the deposit payment. - o 6.6 The payment gateway processes and validates the transaction.

	o 6.7 IF payment succeeds → the system records the deposit as “paid”, submits the request to authorised staff, and displays a confirmation message with the order ID. IF payment fails → go to Alternative Flow C (Payment Failure). 7. IF the selected date is less than 48 hours ahead (Alternative Flow B – Urgent Order): o 7.1 The system marks the request as “urgent”. o 7.2 The request is routed to the Branch Manager for approval. o 7.3 IF the Branch Manager approves → return to step 6.1. IF the Branch Manager rejects → the system notifies the customer that the order cannot proceed and the use case ends.
Alternative Flow A — Design Modification	A.1 The customer is not satisfied with the previewed design. A.2 The customer edits the design details and/or re-uploads the reference image. A.3 The flow returns to step 6.1.
Alternative Flow C — Payment Failure	C.1 The bank/payment gateway returns an authorisation failure. C.2 The system displays a payment error message. C.3 The order remains unconfirmed; the customer may retry payment or cancel.
Alternative Flow D — Unavailable Date	D.1 The selected date/time is unavailable (shop closed or fully booked). D.2 The system displays: “Selected date is not available. Please choose another date.” D.3 The flow returns to step 4.
Post-conditions / Response	• The system has stored the custom cake request with status “pending approval” or “confirmed”. • IF confirmed: the system has recorded the deposit payment status as “paid”, and authorised staff have been notified within 1 minute of submission. • IF not confirmed: the system retains the request in an unconfirmed state with no deposit recorded. • The customer has received a confirmation or rejection message accordingly.
Comments	The customer may place a custom order as a guest without creating an account. Delivery is restricted to Ramallah, Nablus, and Hebron only. A 50% deposit is required before order confirmation (SR5.4). Authorised staff must be notified within 1 minute of submission (SR3.4).

Name: Majd Nazzal

MODIFIED

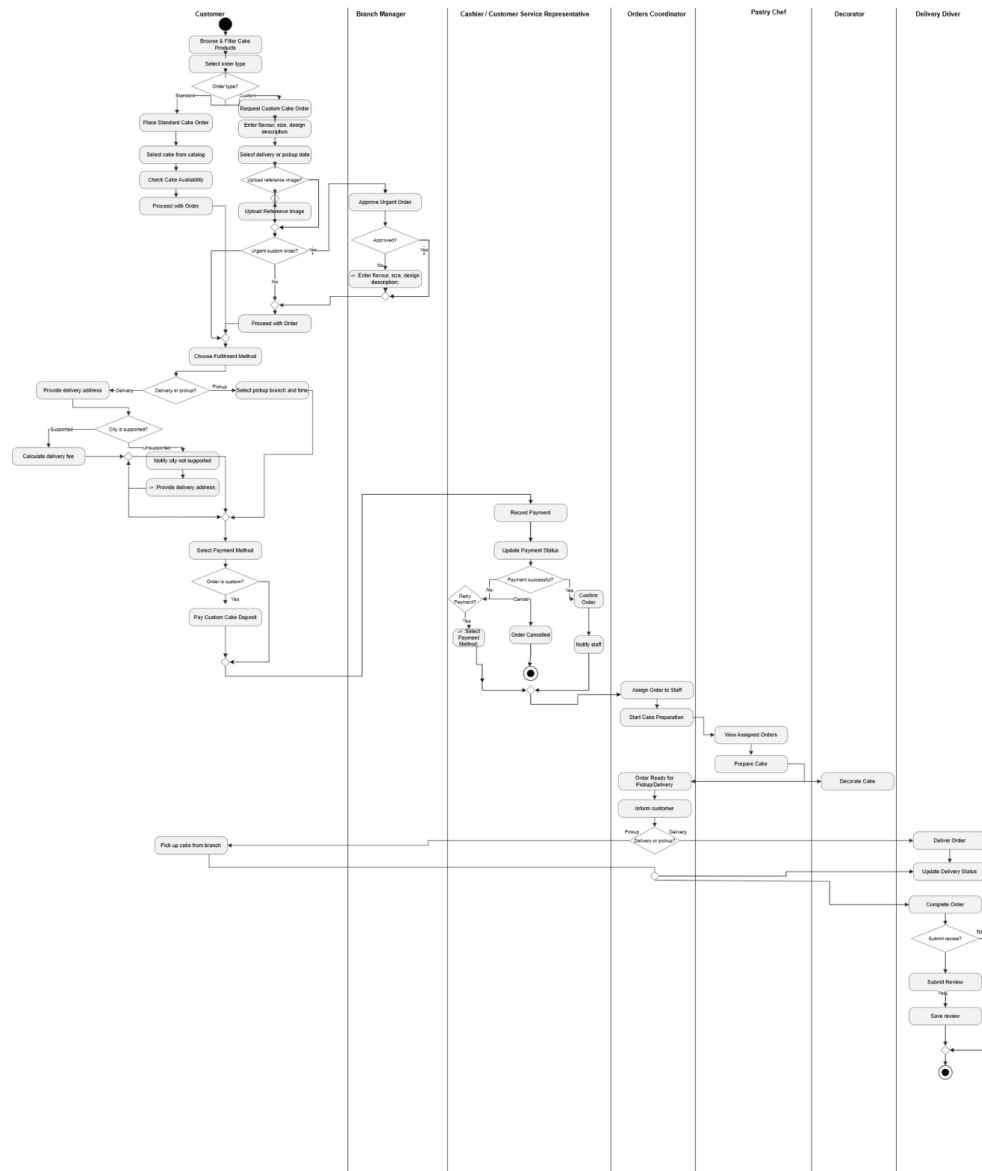
System Name: Cake Shop System	
Use Case Title	Assign Order to Staff
Description	This use case allows the Orders Coordinator to assign a confirmed customer order to an available staff member for preparation, decoration, or delivery. The system records the assignment and updates the order processing status
Actors	Primary Actor: Orders Coordinator
Data	· Order Information · Assignment Information · Staff Availability Information · Order Status
Stimulus / Trigger	Orders Coordinator selects a confirmed order and initiates the staff assignment process.
Pre-conditions	• The Orders Coordinator is logged into the system. • The selected order status is "Confirmed". • The selected order is waiting for staff assignment. • At least one staff member is available for assignment.
Workflow / Sequence of Events	1. The Orders Coordinator opens the list of confirmed orders. 2. The system displays orders with status "Confirmed". 3. The Orders Coordinator selects a specific order. 4. The system displays the order details and required tasks. 5. The Orders Coordinator reviews the order requirements. 6. The Orders Coordinator selects an available staff member for the required task. 7. The system checks staff availability. 8. IF the selected staff member is available THEN 8.1 The system records the assignment.

	8.2 The system changes the order status to "Assigned". 8.3 The system sends a task notification to the assigned staff member. 8.4 The system displays a successful assignment confirmation. ELSE 8.5 The system displays a staff unavailable message. 8.6 The Orders Coordinator selects another staff member. 8.7 The system repeats the availability check.
Alternative / Error Flows	A1. No Available Staff 1. The system detects that no suitable staff member is available. 2. The system displays an availability warning. 3. The Orders Coordinator may postpone the assignment process. 4. The order remains in "Confirmed" status awaiting assignment. E1. Selected Order Already Assigned 1. The system detects that the order has already been assigned. 2. The system displays an appropriate warning message. 3. No new assignment is created.
Post-conditions	Success: • The selected order status is "Assigned". • Assignment information is recorded in the system. • The assigned staff member has received a task notification. • The order is ready for the next processing stage. Failure: • The order remains in "Confirmed" status awaiting assignment. • No assignment record is created.

Comments	Only authorised Orders Coordinators may assign orders. Staff assignments are recorded by the system for order tracking and processing purposes.
----------	---

Contributors: Mohammad Abutteen: drawing], [Sama Baraawi: reviewing, discussion, finalizing],[Nawras Yacoob: reviewing], [Jwan Rajha: reviewing]

MODIFIED by: Mohammad Abutteen



https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&title=cake_shop_activity+%281%29.drawio&dark=auto&fbclid=IwY2xjawSsslJleHRuA2F1bQIxmABicmlkETFCZ1Nza3dOVW5UM2I2b0dGc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHv_-cLKKVFgBASEd3VD3NXtrLYsCnhq0fOE0kkqOqY8rVre1OeejgMHahmsCC_aem_vSYrfGmwkItOXU0EKN0Jg#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1r71ZC0GJzdo1ocHt5bTRV45ecYgNJCyH%26export%3Ddownload#%7B%22pageId%22%3A%22cakeShopActivity%22%7D

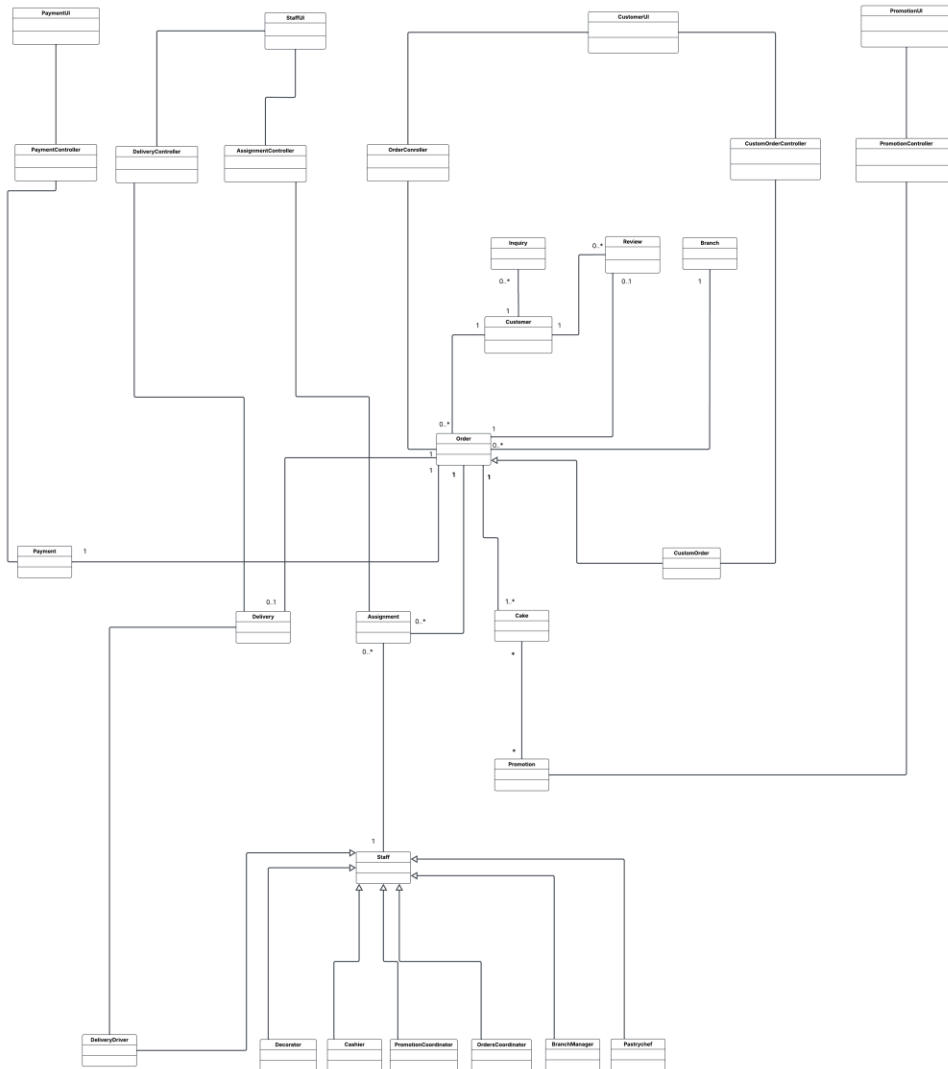
Chapter 3: System Analysis and Modelling

3.1. System CLASS Diagrams

Analysis Class Model:

Lead: Majd Nazzal

Contributors: [Majd Nazzal: drawing], [Sama Baraawi: editing, reviewing], [Nawras Yacoob: discussion], [Jwan Rajha: editing], [Mohammad Abutteen: reviewing]

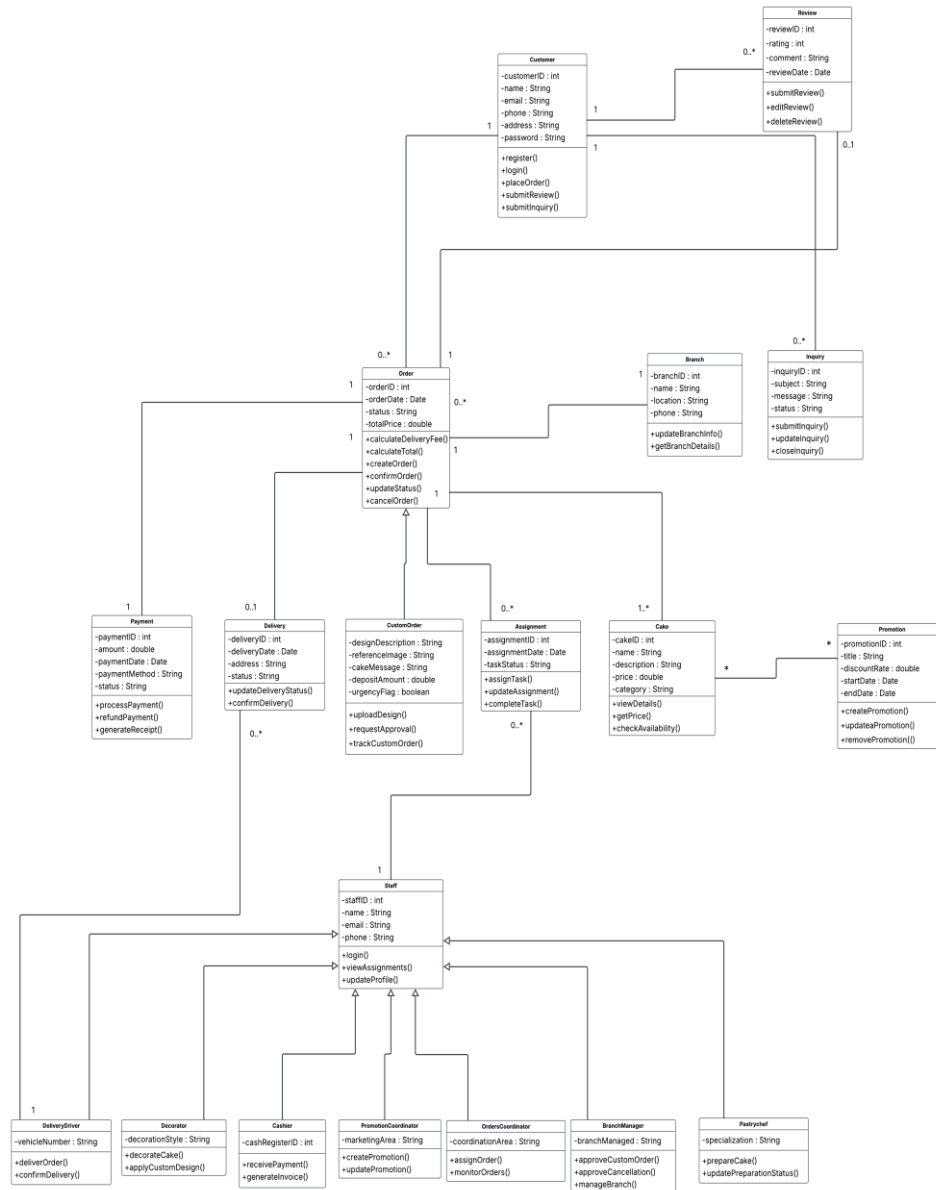


https://lucid.app/lucidchart/971f6c50-edd3-40bf-b9a7-0a3dc1802eeb/edit?invitationId=inv_62131007-171d-4d04-b326-213793a337ea&page=0_0#

Detailed Class Model:

Lead: Majd Nazzal

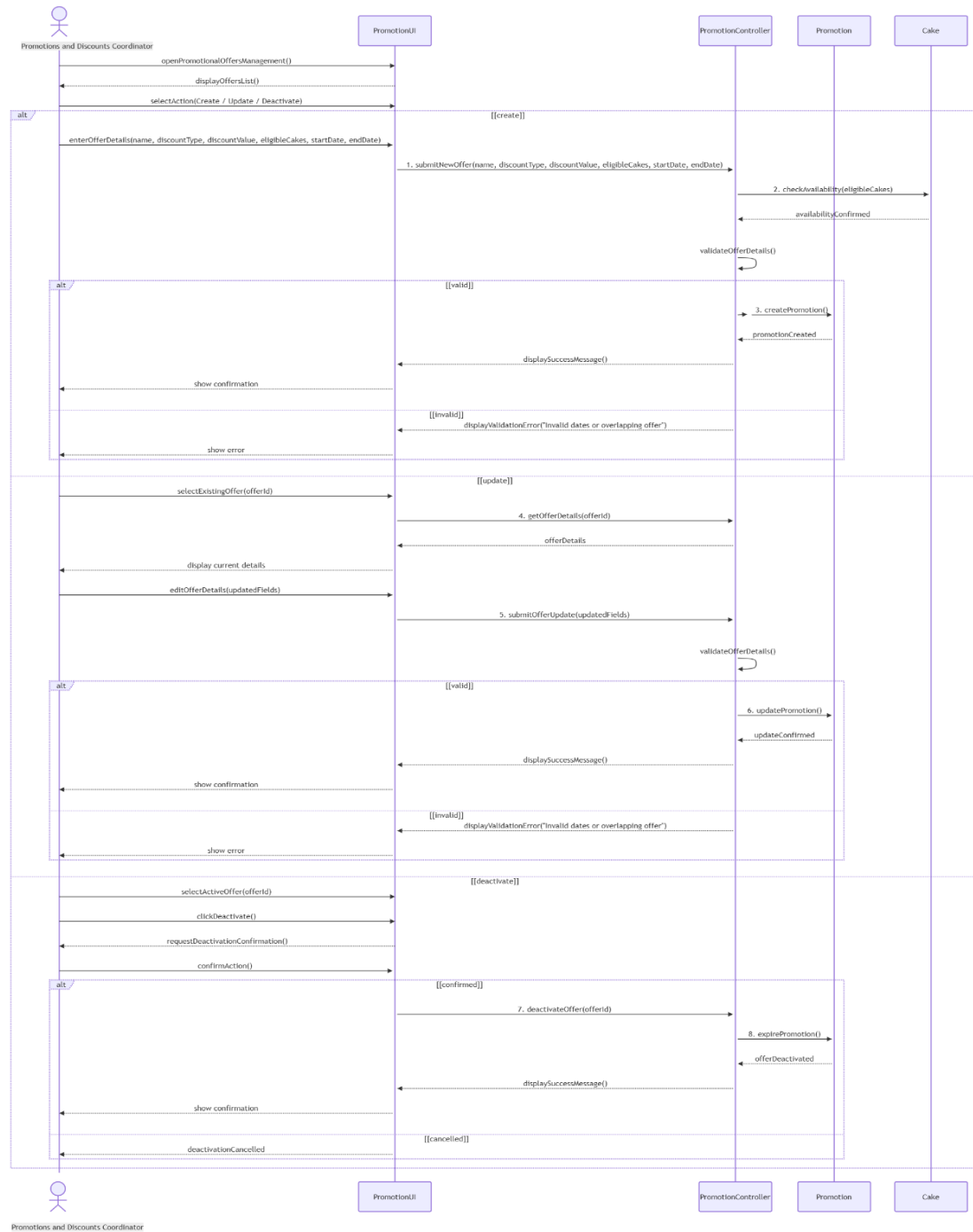
Contributors: [Majd Nazzal: drawing], [Sama Baraawi: editing, reviewing], [Nawras Yacoob: discussion], [Jwan Rajha: editing], [Mohammad Abutteen: reviewing]



https://lucid.app/lucidchart/dbcff3a2-db78-4445-8793-f597f3935f41/edit?invitationId=inv_17f21857-6e0d-4101-ac42-7fdc4bd1190a&page=0_0#

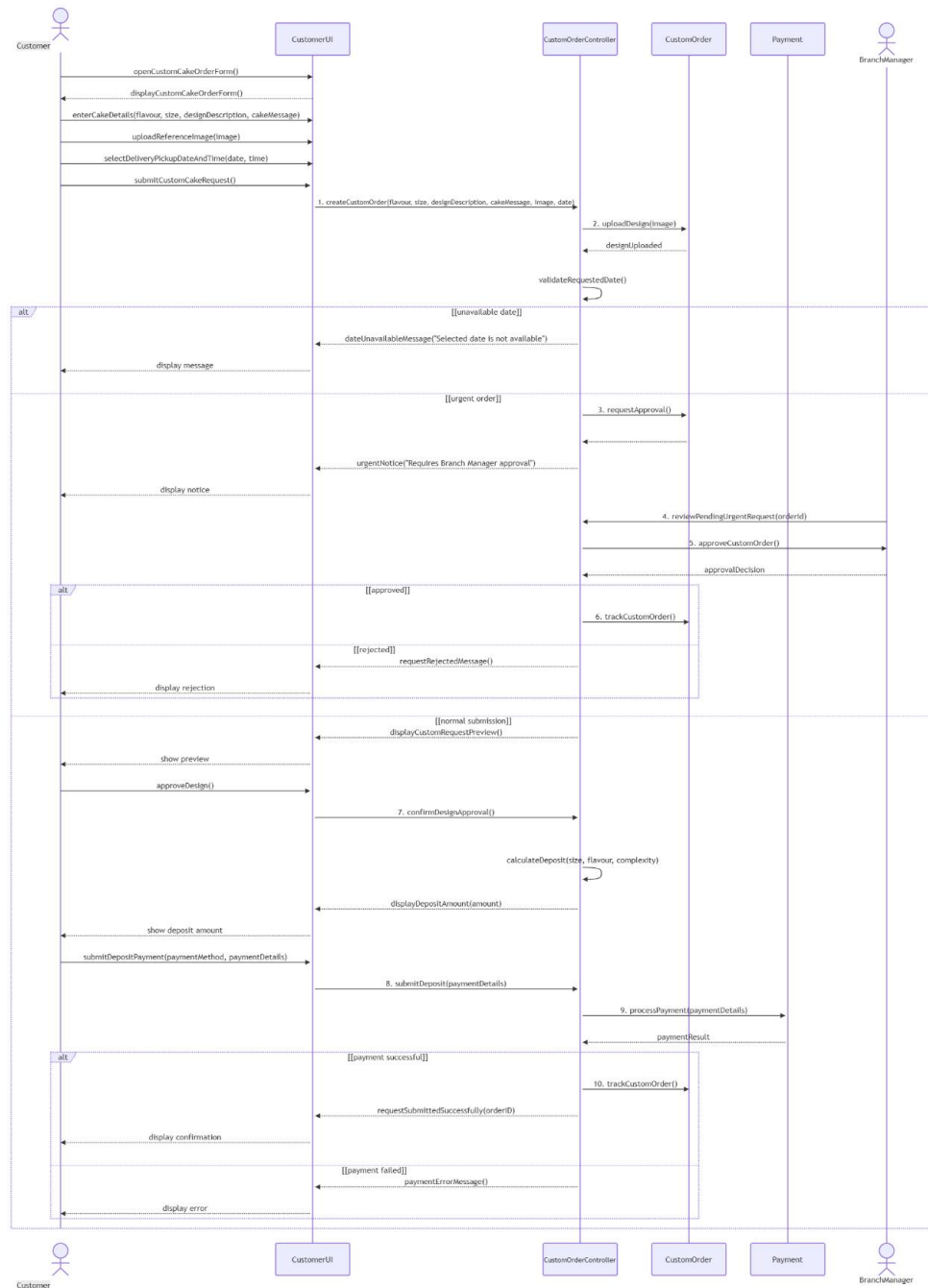
3.2. Sequence Diagrams

Name: Mohammad Abutteen



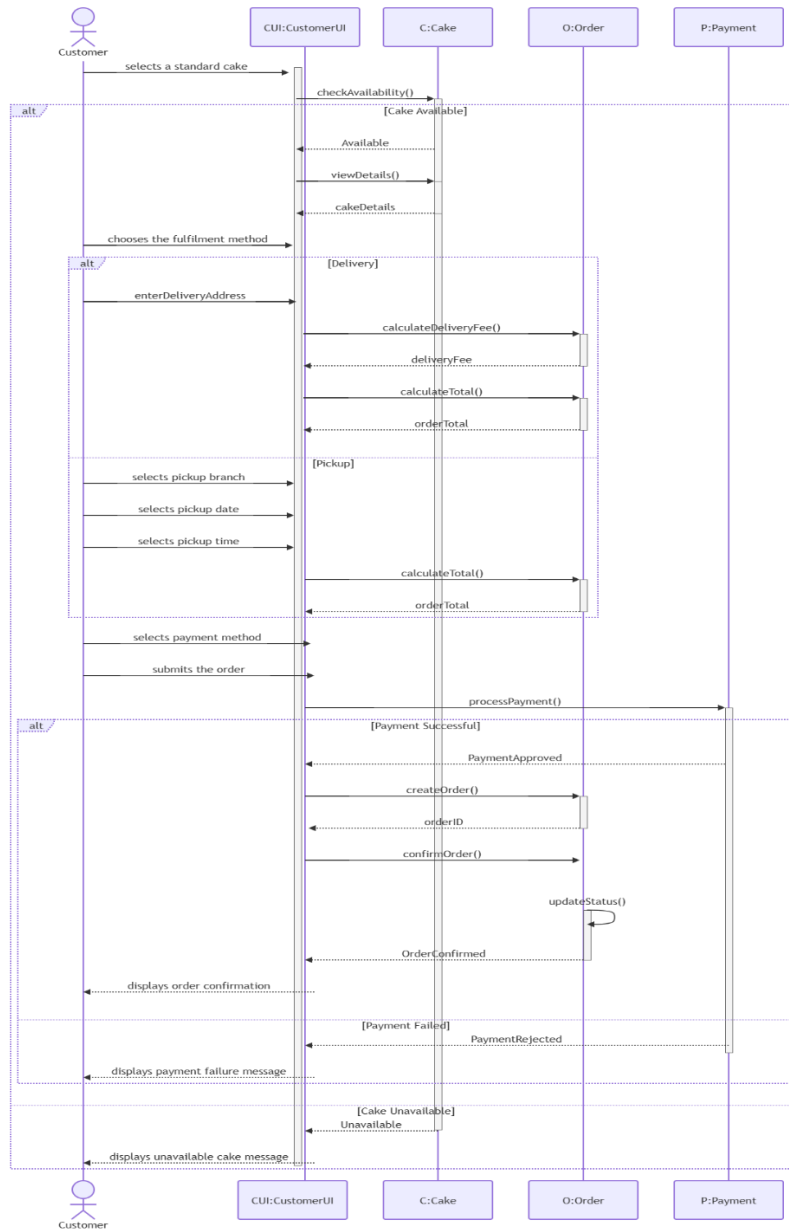
https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1wkEEU9lqtQxFPPXZ-9R_58EQ7UwQUK4z%26export%3Ddownload

Name: Nawras Yacoob



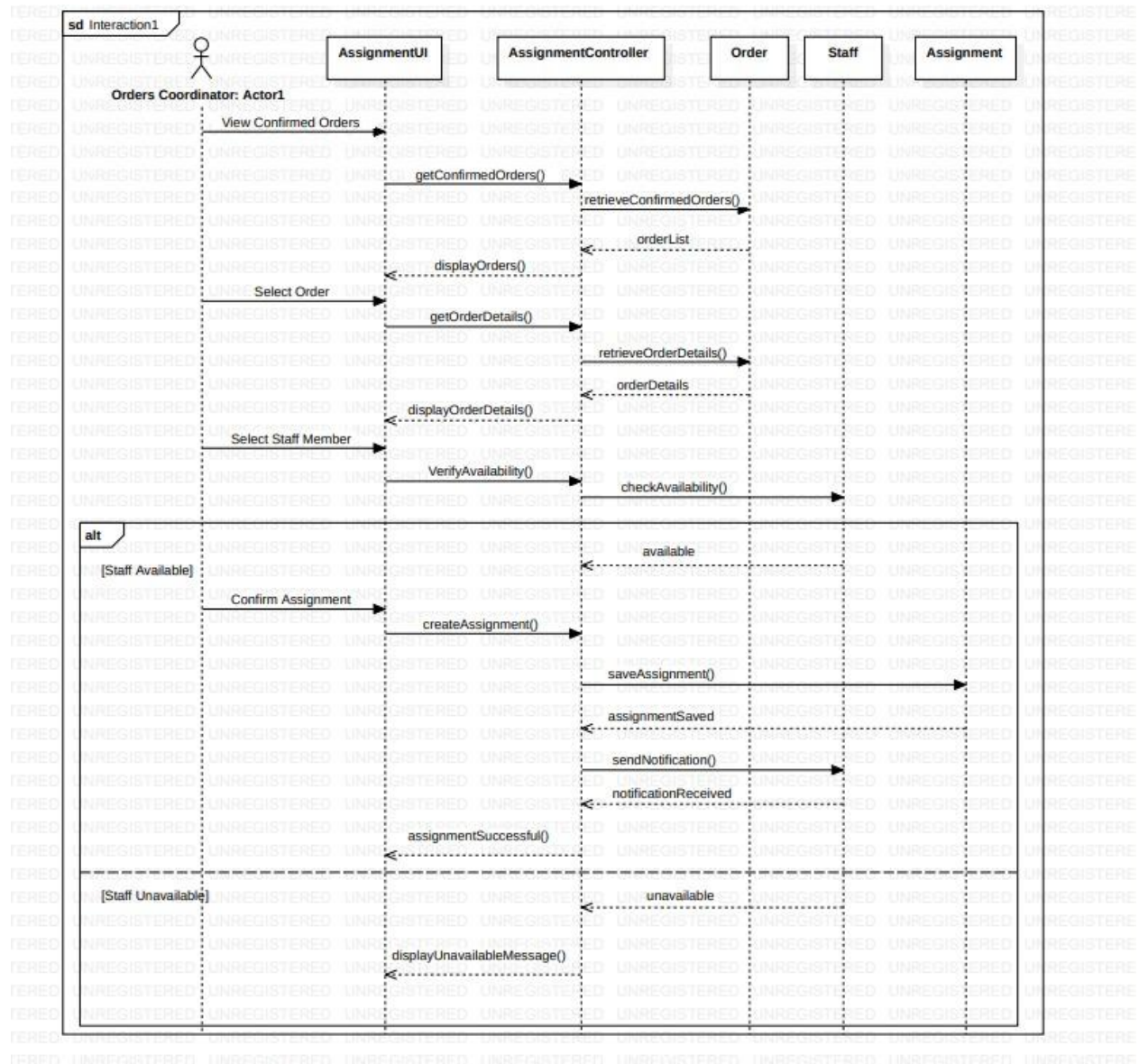
<https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&title=Task4.2%3Asequence%20diagram&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1lic4E1bHjzcbCILBGdeF2xUmEJtpy5xj%26export%3Ddownload>

Name: Sama Baraawi

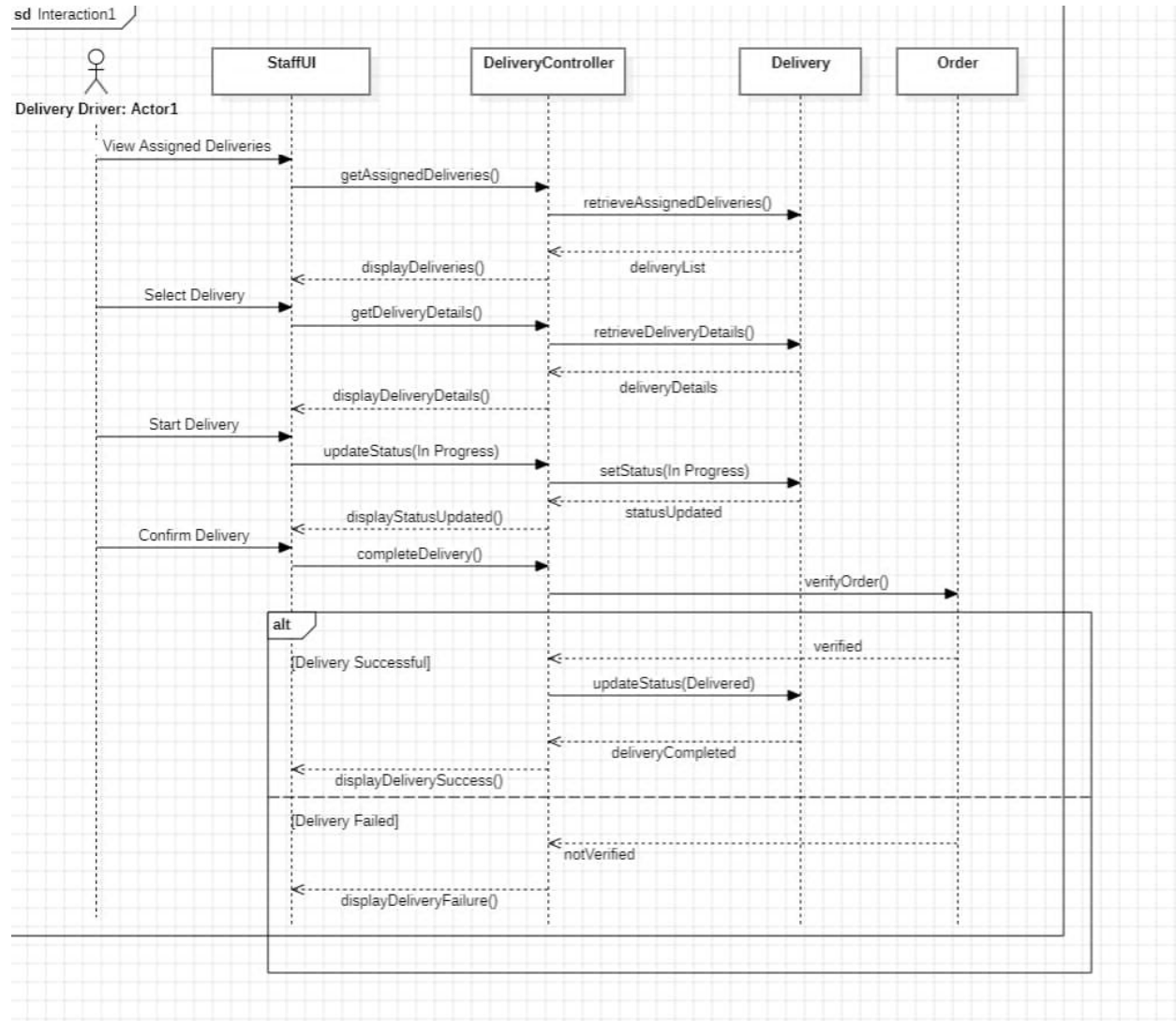


https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1OuDN1k7yIMduUUFoqXdICRda0pNM6_6E%26export%3Ddownload

Name: Majd Nazzal



Name: Jwan Rajha



Chapter 4: System Design and Modelling

4.1. Description of chosen Design Goals

Lead: Sama Baraawi

Contributors: [Sama Baraawi: documentation, reviewing], [Jwan Rajha: discussion], [Nawras Yacoob: reviewing], [Mohammad Abutteen: reviewing], [Majd Nazzal: reviewing]

General Goals:

Design Goal 1: High Cohesion

High cohesion refers to organising the system so that each component is responsible for a single, clearly defined business function. A highly cohesive design improves maintainability, readability, reusability, testing, and future system evolution.

In the Cake Shop System, high cohesion is achieved by dividing the application into specialised business components, each responsible for one business area only. For example, **Order Management** manages order creation and tracking, **Payment Management** handles payment processing, **Delivery Management** manages delivery operations, **Promotion Management** handles promotional activities, **Staff Assignment Management** is responsible for assigning staff, and **Customer Support Management** manages customer enquiries.

This design goal is reflected in the **Component Diagram**, where every component encapsulates its own responsibilities without including unrelated functionality. It is also supported by the **Architecture Diagram**, which separates the system into the UI Layer, Business Logic Layer, and Infrastructure Layer, ensuring that business logic remains organised into cohesive modules. Furthermore, the **Deployment Diagram** preserves this separation by deploying these business components together on the Application Server while maintaining their individual responsibilities.

Design Goal 2: Low Coupling

Low coupling means reducing dependencies between system components so that changes to one component have minimal impact on others. This increases flexibility, maintainability, scalability, and simplifies future system enhancements.

In the Cake Shop System, low coupling is achieved through clearly defined interfaces and layered communication. User Interface components communicate only with the corresponding business components, while business components access infrastructure services through dedicated interfaces rather than directly depending on implementation details. Similarly, external services such as the Payment Gateway are accessed through well-defined service interfaces.

This design goal is clearly demonstrated in the **Architecture Diagram**, where communication occurs only between adjacent layers (UI → Business Logic → Infrastructure), preventing direct access across layers. It is further supported by the **Component Diagram**, where components communicate through provided and required interfaces rather than direct internal connections. Finally, the **Deployment Diagram** reinforces this separation by placing the user interfaces on client devices, business services on the Application Server, the database on a separate Database Server, and the external Payment Gateway on an independent external server, reducing deployment dependencies between system elements.

Specific Goal:

Design Goal 3: Performance

Performance refers to the ability of the system to process requests efficiently and provide fast response times while maintaining reliable operation under normal workloads. Good performance is essential for providing a responsive user experience for both customers and staff.

In the Cake Shop System, performance is achieved through the layered architecture, dedicated application services, and separation of system resources. Business processing is performed on the Application Server, database operations are handled by a dedicated Database Server through the Persistence Service, and payment transactions are processed by an external Payment Gateway. This separation distributes system workload and improves overall response time.

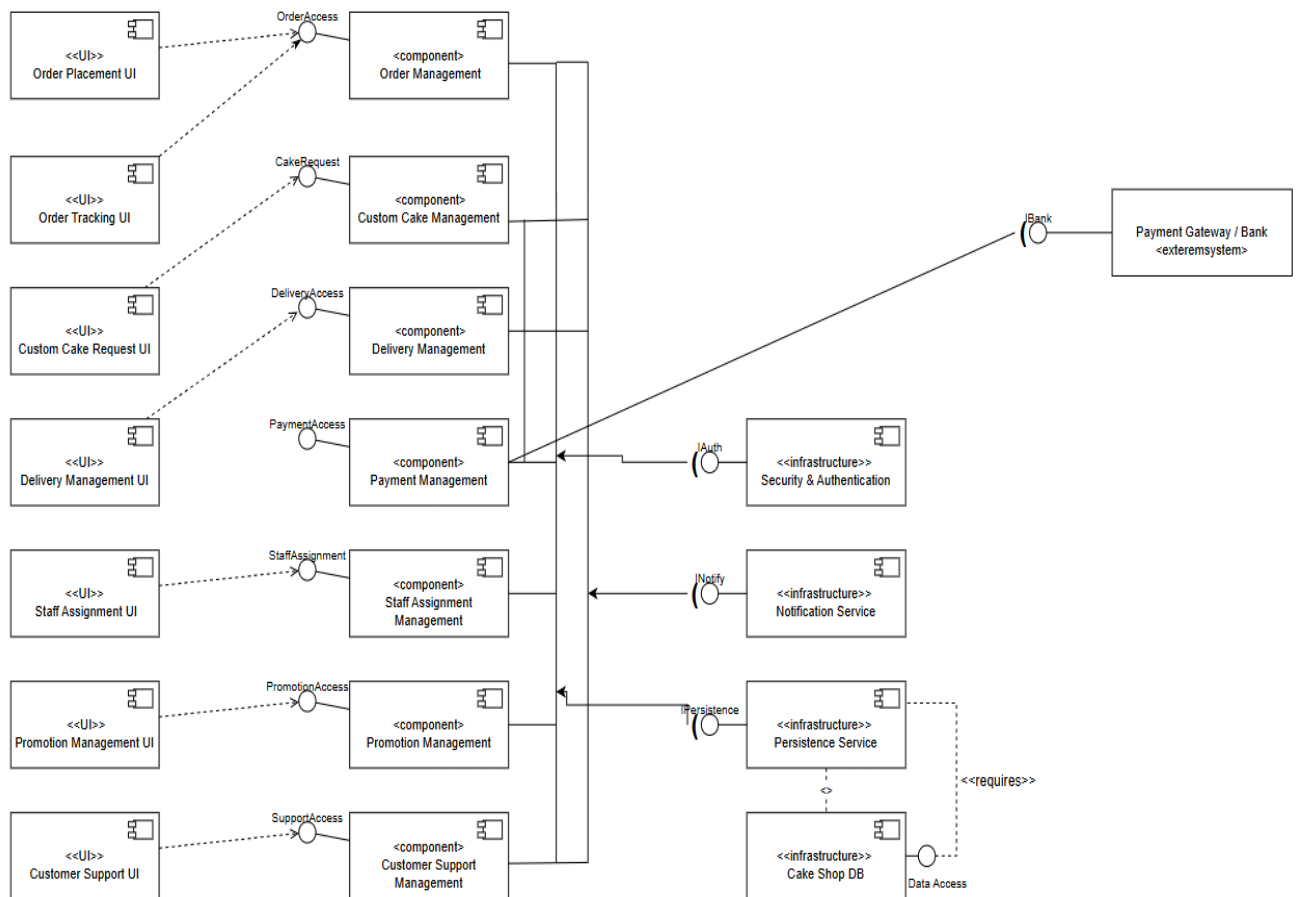
This design goal is supported by the **Deployment Diagram**, where processing responsibilities are distributed across multiple servers rather than concentrated in a single node. It is also reflected in the **Architecture Diagram**, where the Persistence Service provides efficient access to the Cake Shop Database while business services remain independent from data storage implementation.

The system requirements further define measurable performance objectives: product filtering results shall be returned within **three seconds**, and order status updates shall be reflected within **five seconds** after modification by authorised staff. These response-time requirements ensure that customers and employees

4.2. Component Diagram

Lead: Nawras Yacoob

Contributors: [Nawras Yacoob: drawing], [Mohammad Abutteen: redrawing], [Sama Baraawi: discussion, editing], [Jwan Rajha: reviewing], [Majd Nazzal: reviewing]

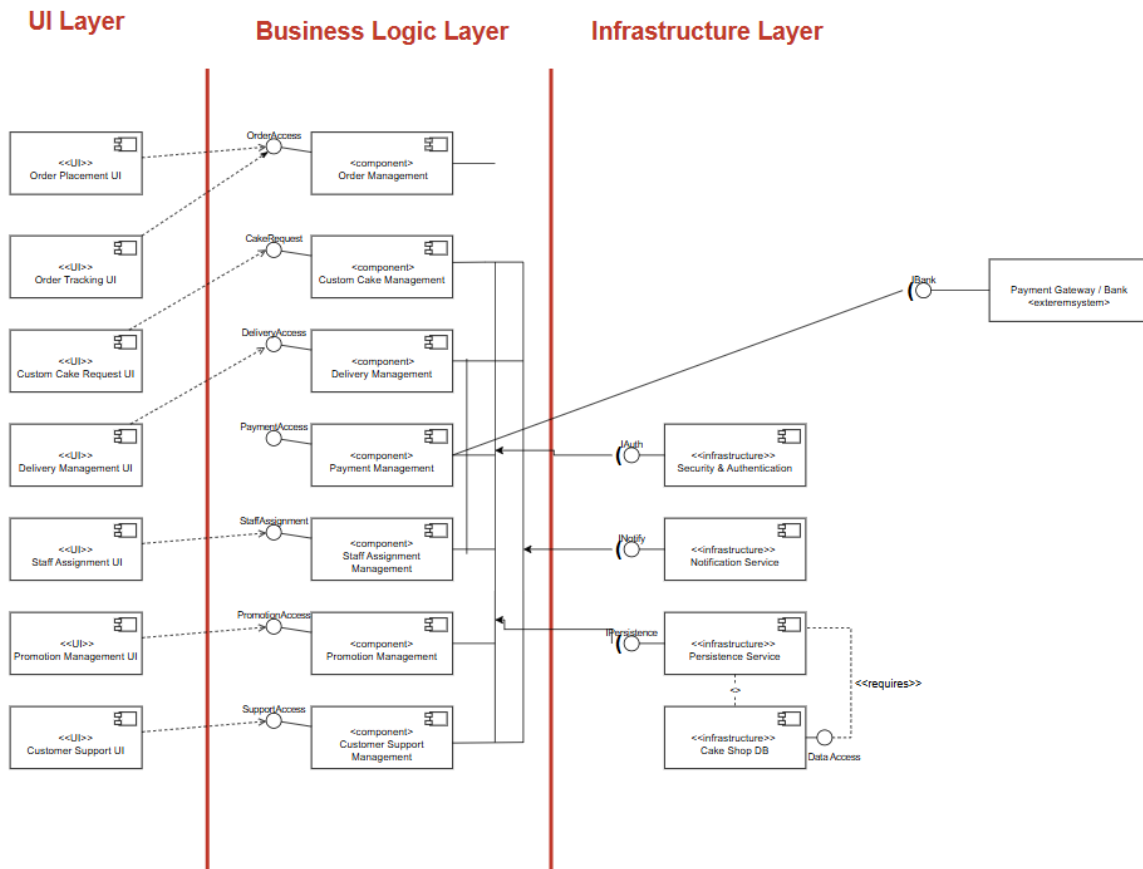


https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&dark=auto&fbclid=IwY2xjawSszuhleHRuA2FlbQlXMAbicmlkETFCZlNza3dOVW5UM2I2b0dGc3J0YwZhchBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHkAzm2Kv4NpL78b-NIMrDF2GsRC9ohf8WzYLU_tjY0p-qHVQFWtlSkwHlId_aem_0JH_alxW1uYjwLMjttVWsQ#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D19MWabPU2nKRY4DS-kFvI004eu_LLzpX_%26export%3Ddownload%2F%7B%22pageId%22%3A%22cakeShopComponent%22%7D

4.3. Overall architecture diagram

Lead: Mohammad Abutteen

Contributors: [Mohammad Abutteen: drawing], [Nawras Yacoob: redrawing], [Sama Baraawi: editing, reviewing, documentation], [Jwan Rajha: reviewing], [Majd Nazzal: reviewing]



https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&title=cake_shop_component.drawio&dark=auto&fbclid=IwY2xjawSswBJleHRuA2FlbQIxMABicmlkETFCZ1Nza3dOVW5UM2I2b0dGc3J0YwZhcHBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHizZbIsuBdQSV2C3C-HGFf74NTt9vaDmWbuPf8bTnph-5p3rApOoqMflls_2_aem-M8ITTRAVNrt3g6ljEH-yA#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1OPVW_VJbvElnOMjZlw0Nk3TfLghZpLWc%26export%3Ddownload%7B%22pageId%22%3A%22cakeShopComponent%22%7D

The Cake Shop System adopts a **Layered Architecture**, consisting of three main layers: the **UI Layer**, **Business Logic Layer**, and **Infrastructure Layer**. This architectural style was selected because it provides a clear separation of responsibilities, making the system easier to develop, maintain, test, and extend.

The **UI Layer** contains all user interface components that interact with customers and staff, including Order Placement UI, Order Tracking UI, Custom Cake Request UI, Delivery Management UI, Staff Assignment UI, Promotion Management UI, and Customer Support UI. This layer is responsible only for collecting user input and presenting system information without containing any business logic.

The **Business Logic Layer** contains the core application components that implement the business rules of the Cake Shop System. These include Order Management, Custom Cake Management, Delivery Management, Payment Management, Staff Assignment Management, Promotion Management, and Customer Support Management. Each component is responsible for a specific business function and communicates with other services only through well-defined interfaces.

The **Infrastructure Layer** provides common technical services required by the business layer. It includes the Security & Authentication component for user authentication and access control, the Notification Service for sending system notifications, the Persistence Service for handling database operations, the Cake Shop Database for data storage, and the external Payment Gateway for processing online payments.

The Layered Architecture was chosen because it directly supports the system's design goals. **High cohesion** is achieved by assigning each business component a single, clearly defined responsibility. **Low coupling** is achieved by separating the presentation, business, and infrastructure concerns and allowing communication only through well-defined interfaces between adjacent layers. This architecture also supports the system's **performance** requirements by separating business processing from database access and external payment services, allowing each layer to perform its tasks efficiently while reducing unnecessary dependencies.

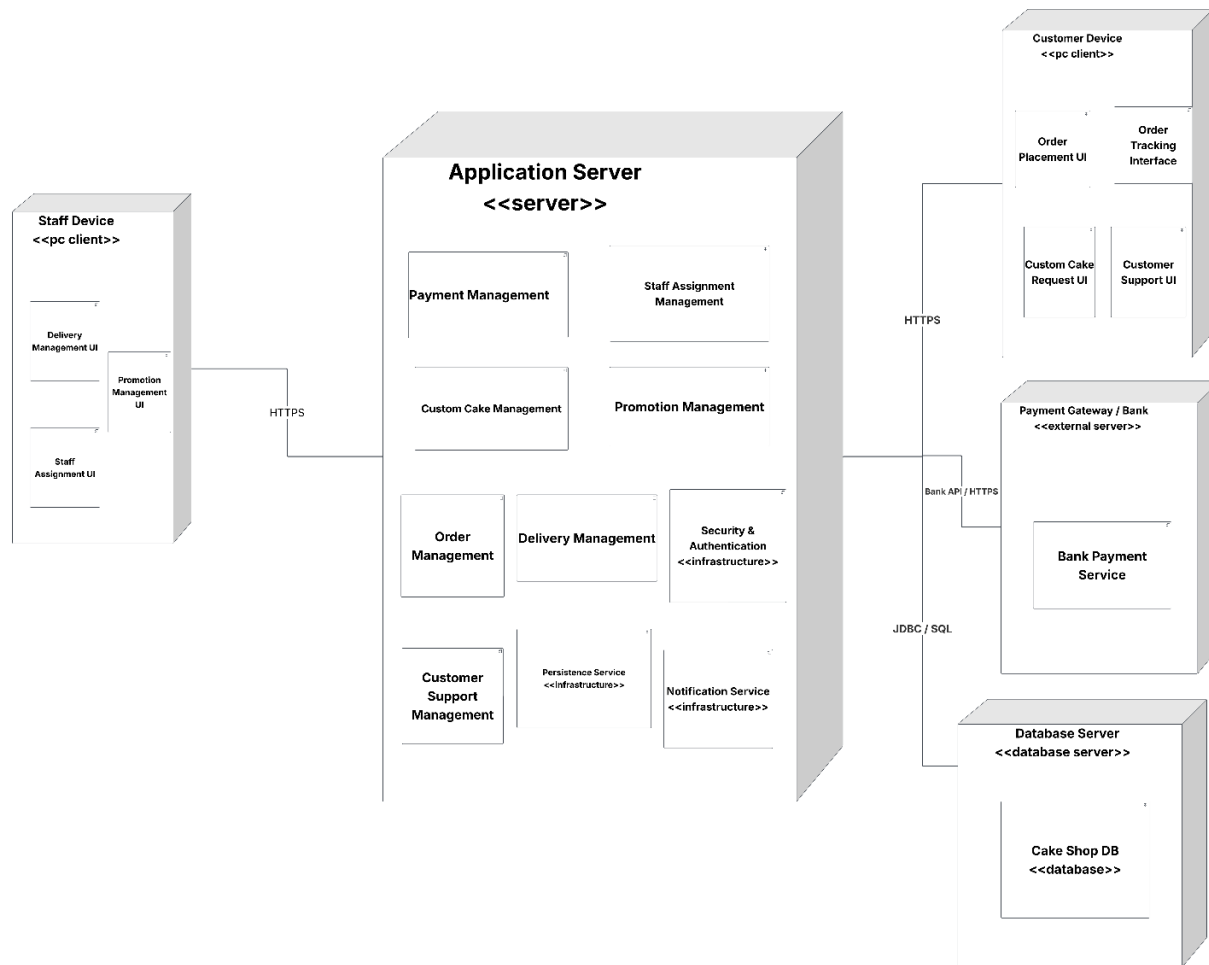
Furthermore, this architecture improves maintainability and scalability by allowing new business features, additional services, or external integrations to be added with minimal impact on existing components. It also enhances system reliability by isolating technical services, such as authentication, notifications, persistence, and payment processing, from the core business logic, making the overall system easier to maintain and evolve as business requirements change.

4.4. Deployment diagram

Lead: Jwan Rajha

Contributors: [Jwan Rajha: drawing], [Sama Baraawi: redrawing, discussion], [Majd Nazzal: editing], [Nawras Yacoob: reviewing], [Mohammad Abutteen: reviewing]

Task 4.6 - Deployment Diagram (Cake Shop System)



<https://lucid.app/lucidspark/77b17dce-e537-45d5-8199-5b576649b174/view>

Appendix-I: Minutes of Meetings

Throughout the project, our team held regular meetings after the release of each project phase to ensure effective planning, coordination, and communication among all members. Depending on everyone's availability, these meetings were conducted either online or face-to-face. Typically, the first meeting after a new phase was released focused on reviewing the project requirements, understanding the expected deliverables, clarifying any ambiguous points, distributing responsibilities, and agreeing on a clear implementation plan and timeline.

As each member progressed with their assigned tasks, we held one or more follow-up meetings to present our work, explain our diagrams and documentation, exchange ideas, and receive feedback from the rest of the team. During these meetings, we carefully reviewed each deliverable together, discussed possible improvements, resolved design issues, and ensured consistency across all project artifacts. Whenever necessary, diagrams and documents were revised based on the team's discussions and the instructor's feedback.

Before every submission deadline, we conducted a final review meeting to verify that all required deliverables had been completed, formatted consistently, and fully complied with the project requirements. We also checked that the different diagrams and documentation were logically connected and free from inconsistencies before submitting the final version.

In addition to our internal meetings, we also conducted meetings with the customer, as documented in the previous project phases. The initial customer meeting was held to gather and clarify the system requirements, understand the customer's expectations, and define the project scope. After refining and updating the requirements based on our analysis and discussions, we held a follow-up meeting with the customer to review the revised requirements and confirm key project constraints, including the estimated timeline and project cost. These meetings ensured that the final requirements accurately reflected the customer's needs and expectations.